UNIVERSIDAD
DE SANTIAGO
DE CHILE

Informe Laboratorio N°1

Manipulador Robótico

Asignatura: Fundamentos de Robótica – Departamento de Ingeniería Mecánica, USACH

Profesor: Diego Luna; Arturo Álvarez

Integrantes:

- Martin Barrezueta
- Ignacio Espinoza
- Emilio Rojas

Fecha: 06-11-2026

Resumen

En este laboratorio se implementó un manipulador robótico de 4 grados de libertad (GDL) fabricado mediante impresión 3D, accionado por servomotores y controlado por una placa ESP32. El robot es capaz de detectar bloques cúbicos en un punto conocido del área de trabajo mediante un sensor ultrasónico HC-SR04, trasladarlos hasta un punto final y apilarlos. El posicionamiento del elemento terminal se resolvió con el modelo de Denavit–Hartenberg (D-H): se obtuvo la cinemática directa, y a partir de ella la cinemática inversa en forma analítica, lo que permite calcular los ángulos de cada articulación para una posición cartesiana deseada. Se desarrolló además una aplicación móvil Android que, por Bluetooth, permite control manual, ejecución de rutinas automáticas y visualización de las posiciones articulares calculadas. La alimentación se separó en dos etapas (potencia para los servos y control para la ESP32) compartiendo únicamente la referencia de tierra (GND).

1. Introducción

Los manipuladores robóticos combinan tecnologías electromecánicas, sistemas de automatización y herramientas de software, lo que les permite realizar tareas que requieren precisión y repetibilidad. Por esta razón, son ampliamente utilizados en distintos sectores industriales, como el automotriz, alimenticio, aeroespacial y médico. Entre sus aplicaciones más habituales se encuentra el traslado y la organización de objetos.

En este laboratorio se desarrolla una implementación a pequeña escala, utilizando un brazo robótico impreso en 3D. El sistema tiene como objetivo detectar bloques ubicados en posiciones previamente establecidas y trasladarlos para apilarlos en un punto determinado, empleando un modelo matemático que permita calcular y controlar la posición de la pinza del manipulador.

1.1. Objetivo general

Implementar un robot manipulador a escala, impreso en 3D, capaz de trasladar objetos y ordenarlos mediante servomecanismos controlados con un controlador.

1.2. Objetivos específicos.

1. Construir el manipulador con piezas impresas en 3D, uniendo eslabones y fijaciones mediante pernos y tornillos.

2. Integrar el hardware: actuadores (servomotores), controlador (ESP32), sensor ultrasónico y comunicación Bluetooth.

3. Implementar control manual y automático desde una aplicación móvil Android.

4. Modelar el manipulador mediante el algoritmo de Denavit–Hartenberg y usar la cinemática inversa para posicionar el elemento terminal.

5. Ejecutar rutinas automáticas de traslado y apilamiento de objetos cúbicos de dimensiones conocidas.

2. Marco teórico

- **Manipuladores robóticos:** los manipuladores están compuestos por una cadena cinemática de cuerpos rígidos (eslabones) y unido por articulaciones. Cada articulación aporta un grado de libertad (GDL), y el conjunto determina la posición y la orientación del elemento terminal [1][2]. En este trabajo, la base, el hombro y el codo posicionan la pinza, y un cuarto servomotor acciona su apertura.

- **Método Denavit-Hartenberg y cinemática inversa:** la cinemática directa calcula la posición del elemento a partir de los ángulos de las articulaciones y la cinemática inversa obtiene los ángulos para alcanzar una posición dada [1][2]. La convención de Denavit-Hartenberg describe cada eslabón con cuatro parámetros: el ángulo de articulaciones θ, la distancia d a lo largo del eje z, la longitud *a* en el eje x y el ángulo de torsión α [1]. La transformación homogénea de un eslabón es:

$$T(\theta, d, a, \alpha) = \begin{bmatrix} \cos\theta & -\sin\theta\,\cos\alpha & \sin\theta\,\sin\alpha & a\,\cos\theta \\ \sin\theta & \cos\theta\,\cos\alpha & -\cos\theta\,\sin\alpha & a\,\sin\theta \\ 0 & \sin\alpha & \cos\alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

- **Servomotores:** un servo motor posiciona su eje mediante una señal PWM, cuyo periodo suele ser de 20 ms (50 Hz), el recorrido completo corresponde a pulsos de entre 0,5 y 2,5 ms según el modelo [6]. Los modelos SG90 y MG90S se diferencian en el torque, material de los engranajes, peso y consumo. El torque para un SG90 es de 1,8 kg·cm y para el MG90S es de 2,2 kg·cm [6].

- **Driver PWM PCA9685:** El PCA9685 es un controlador de 16 canales PWM de 12 bits (4096 pasos por periodo), con interfaz I2C y frecuencia configurable [3]. A 50 Hz, cada paso equivale a unos 4,9 µs, lo que permite posicionar varios servomotores con una sola conexión de dos hilos.

- **Controlador ESP32:** La ESP32 es un microcontrolador de doble núcleo con Wi-Fi y Bluetooth. Sus pines trabajan a 3,3 V y no toleran 5 V [4].

- **Sensor ultrasonico HC-SR04:** El HC-SR04 mide la distancia por tiempo de vuelo, un pulso de 10 μs en TRIG dispara una rafaga ultrasónica y el pulso resultante tiene en ECHO tiene un ancho igual al tiempo de ida y vuelta. La distancia es $d = \frac{t \cdot v}{2}$,

con $v \approx 343\, m/s$ a $20^{\circ} C$ [5]. El sensor opera a 5V, por lo que requiere un divisor de voltaje para conectarse a la ESP32.

- **Alimentación:** los servomotores pueden generar picos de consumo de corriente durante el arranque o cuando realizan esfuerzos, provocando variaciones en el voltaje e interferencias. Un cargador de celular entrega 5V regulados y puede alimentar directamente a los servos. La ESP32 se alimenta por separado desde el puerto USB del PC, asi para que el ruido de los servos no desconfigure al controlador y se mantiene una tierra común entre ambas fuentes.

- **Comunicación y aplicaciones:** El Bluetooth clásico con perfil serie (SPP) permite enviar texto entre un teléfono y la ESP32. La aplicación móvil se desarrolla en MIT App Inventor [7].

3. Presentación del problema

Se debe implementar un brazo robótico que se posicione utilizando el modelo Denavit-Hartenberg y ubique su elemento terminal (garra) en distintas posiciones, para trasladar objetos desde un punto inicial hasta dejarlos apilados en un punto final. Los requerimientos son:

- Emular el traslado de objetos de dimensiones y pesos conocidos desde puntos específicos del área de trabajo.
- Detectar la existencia de bloques en esos puntos conocidos mediante el sensor ultrasónico.
- Ejecutar una rutina que traslade los bloques desde el punto detectado Qi hasta otro punto Qf, de modo que queden apilados.
- Disponer de una aplicación móvil que permita la interacción entre el manipulador y el controlador.
- Visualizar las posiciones de las articulaciones determinadas mediante el algoritmo D-H.
- Ejecutar rutinas automáticas con objetos cúbicos de dimensiones conocidas y soportables por el manipulador.

Condiciones del problema. Cubo de PLA de 15 mm de arista. Posiciones de recogida (centro del cubo, en mm, respecto de la base): P1 (165,5; 0), P2 (150; 70) y P3 (106,4; 126,8). Punto de depósito Qf: (28,7; −163). Las cuatro posiciones están a 165,5 mm del eje de la base, y el alcance máximo teórico del punto de agarre desde el eje del hombro es de 221,6 mm.

4. Solución del problema

El sistema se organiza en capas: interfaz de usuario (aplicación), comunicación (bluetooth), control de movimiento (cinemática D-H), drives (servos, sensor y garra) y hardware (ESP32, actuadores, servos y fuentes).

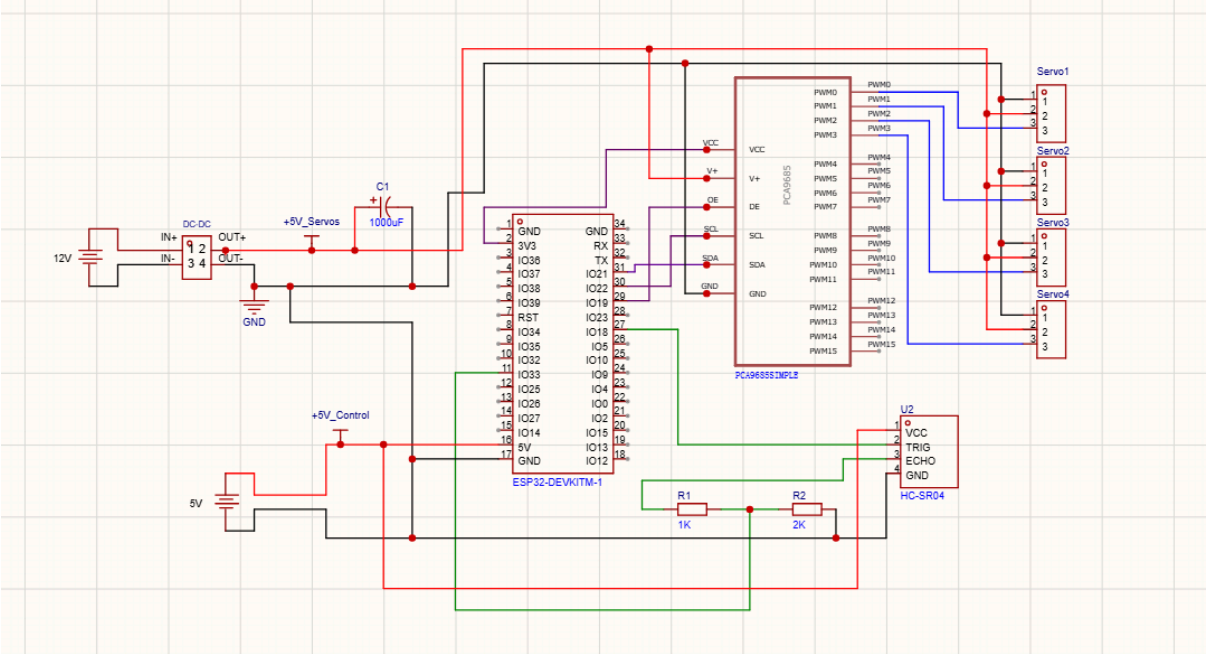
Diseño

El manipulador se fabricó en PLA mediante impresión 3D, uniendo los eslabones con pernos y tornillos. Las longitudes de los eslabones: el eje del hombro está a 129,5 mm sobre la mesa, el brazo mide 70,53 mm y el antebrazo 50,035 mm. El punto de agarre queda 100,3 mm por delante de la muñeca y 15 mm por debajo de la línea del antebrazo.

Hardware

- Modulo ESP-32D: controlador principal y bluetooth, lógica de 3,3V alimentado desde el PC.
- PCA9685: driver PWM de 16 canales, se usaron los canales del 0 al 3. La señal SDA y SCL se conectaron a IO21 e IO22 de la ESP32 respectivamente. El VCC se conectó al 3V3 de la ESP32.
- Servomotores: 2 servomotores SG90 para la base y la garra, y 2 servomotores MG90S para el hombro y el codo. Se conectó la señal PWM a la PCA, y también se conectó directamente a la fuente.
- HC-SR04: sensor ultrasónico para la detección de bloques. Se conectó TRIG a IO18 y ECHO a IO33 a través de un divisor de voltaje donde se usó resistencias de 1Ω y 2Ω.
- Cargador de celular de 5V: alimenta los servomotores.
- Condensador: Estabiliza la línea de los servos, se conecto entre la fuente de energía y los servos.

La alimentación se divide en dos fuentes independientes: potencia (cargador de celular de 5 V , que alimenta los servomotores) y control (puerto USB del PC), que alimenta la ESP32 y desde su pin VIN, el HC-SR04. La lógica del driver (VCC del PCA9685) se alimenta a 3,3 V desde la ESP32. Como el cargador y el PC son fuentes independientes, sus GND se conectan entre sí, las líneas de +5 V de ambas fuentes nunca se unen.



Matriz de transformación

Siguiendo las instrucciones del método de Denavit-Hartenberg se desarrolló la matriz que explica cómo las coordenadas de la garra cambian según los cambios de ángulo de los servomotores.

$$\begin{bmatrix} Cq_1\left(Cq_2Cq_3 - Sq_2Sq_3\right) & -Cq_1\left(Cq_2Sq_3 + Sq_2Cq_3\right) & -Sq_1 & -L_1Sq_1 - L_3Sq_1 \\ Sq_1\left(Cq_2Cq_3 - Sq_2Sq_3\right) & -Sq_1\left(Cq_2Sq_3 + Sq_2Cq_3\right) & Cq_1 & L_2Cq_1 + L_3Cq_1 \\ -Cq_2Sq_3 - Sq_2Cq_3 & Sq_2Sq_3 - Cq_2Cq_3 & 0 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

S: Acrónimo para el seno

C: Acrónimo para el coseno

q_1: Cambio del ángulo para el servo 1

q_2: Cambio del ángulo para el servo 1

q_3: Cambio del ángulo para el servo 1

L_1: Largo de la extremidad 1

L_2: Largo de la extremidad 2

L_3: Largo de la garra

Cinemática inversa

Debido a la complejidad de la matriz de transformación nos vimos forzados a desarrollar las ecuaciones para determinar el ángulo necesario según las coordenadas objetivo a mano. Para facilitar su explicación y aplicación en el código del brazo se hizo uso de ecuaciones complementarias.

Ecuaciones principales
$q_1 = \tan^{-1}\left(\frac{y}{x}\right)$
$q_2 = 90 - (\alpha + \beta)$
$q_3 = \alpha + \beta$

Ecuaciones complementarias
$l = \sqrt{x^2 + y^2}$
$h = \sqrt{l^2 + z^2}$
$\alpha = \tan^{-1}\left(\frac{z}{l}\right)$
$\beta = \cos^{-1}\left(\frac{L_2^2 + h^2 - L_3^2}{2L_2 h}\right)$

Software de control

- Configuración: pines, dirección I2C del PCA9685, geometría, límites de cada servo, parámetros de pulso PWM y velocidades.
- Driver de servos: convierte un ángulo (0 a 180°) en el valor de pulso del PCA9685, a 50 Hz.
- Control de movimiento: cada movimiento se ejecuta de forma gradual y sincronizada: la articulación de mayor recorrido avanza 1° por paso y las demás llegan al mismo tiempo. Los desplazamientos de la garra siguen una trayectoria en arco con puntos intermedios cada 2 mm, calculados con la cinemática inversa. La pausa entre pasos es de 15 ms sin carga y 25 ms con carga.
- Rutina pick and place: la pinza recorre las tres posiciones, mide con el sensor en cada una y, si detecta un cubo, lo recoge, lo transporta y lo deposita en la pila, subiendo un nivel de 15 mm por cada cubo apilado. Al terminar vuelve a la posición de inicio.
- Detección: el sensor ultrasónico determina si hay un bloque cuando la distancia medida es menor o igual que un umbral de 10 cm; cada medición es la mediana de 5 lecturas.

5. Conclusiones

Se logró cumplir el objetivo general del proyecto construyendo un manipulador robótico, fabricado con impresión 3D y controlado por una ESP32, detecta cubos en tres posiciones conocidas y lo traslada hasta apilarlos.

Se desarrolló un manipulador de cuatro articulaciones, controlado por una ESP32 y un módulo PCA9685, que permite detectar los cubos mediante un sensor ultrasónico, recogerlos y trasladarlos hasta el punto de apilamiento. Además, se utilizó el D-H para calcular los ángulos de los servomotores correspondientes a cada posición de la rutina, permitiendo ubicar la garra sobre los cubos y posicionarla para realizar el apilamiento.

También se implementó un sistema de alimentación independiente para los servomotores y la ESP32, utilizando un cargador de 5V para los servos y el puerto USB del PC para el controlador, manteniendo una tierra común.

El torque de los servos y el peso de los eslabones fueron el factor limitante del alcance y de la carga manejable. En una prueba de esfuerzo un servo SG90 no tuvo torque suficiente y otro se dañó, por lo que se evaluaron servos de engranajes metálicos en las articulaciones más cargadas.

Por último, se realizó una aplicación para controlar el brazo robótico, para facilitar la operación y permitiendo visualizar los angulos en tiempo real.

6. Referencias

1. *Su, H. (2026). Robot Kinematics (apuntes del curso). The Ohio State University. https://cdn.jsdelivr.net/gh/haijunsu-osu/lec_robot_kinematics@main/robot_kinematics.pdf*
2. *1. Asada, H., y Leonard, J. (2005). 2.12 Introduction to Robotics, cap. 4: Planar Kinematics. MIT OpenCourseWare. https://ocw.mit.edu/courses/2-12-introduction-to-robotics-fall-2005/resources/chapter4/*
3. *NXP Semiconductors. PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller (hoja de datos).*
4. *Espressif Systems. ESP32 Series Datasheet.*
5. *Hoja de datos del sensor ultrasónico HC-SR04.*
6. *Hojas de datos de los servomotores SG90, MG90S*
7. *MIT App Inventor. Documentación. https://appinventor.mit.edu*

7. Anexo I: rutina de código

https://github.com/EmRosess/brazo-robotico.git

1.1 Descripción de la rutina

La función ejecutarRutinaApilado() revisa tres posiciones con el sensor HC-SR04 y apila en una pila fija los cubos que encuentra. El brazo (ESP32 + PCA9685, 3 articulaciones y pinza) se mueve siempre por puntos cartesianos: cada punto se convierte en ángulos de servo con la cinemática inversa del Anexo 2.

Secuencia de la rutina:

1. Validar la configuración y comprobar que todos los puntos y tramos son alcanzables antes de mover nada (configuracionLista() y validarRutina()).
2. Abrir la pinza e ir a la pose de escaneo de la posición 1.
3. En cada posición, medir la distancia (mediana de 5 lecturas). Hay objeto si el umbral es mayor o igual que la distancia medida.
4. Si hay objeto, transportarObjeto(): bajar, cerrar la pinza, subir, ir sobre la pila, bajar al nivel correspondiente, soltar y subir libre. Si no hay objeto, se omite la posición.
5. Pasar a la pose de escaneo de la siguiente posición.
6. Al terminar, volver a la pose de inicio y devolver RUTINA_OK.

Si llega STOP o un punto no es alcanzable, la rutina se interrumpe y devuelve RUTINA_DETENIDA, RUTINA_ALCANCE o RUTINA_ERROR.

1.2 Parámetros de la rutina

Valores del archivo Brazo_Apilado_Cinematica_ESP32_v4_wifi.ino.

Parámetro	Valor	Uso
CANTIDAD_POSICIONES	3	Posiciones de recogida que se escanean
POSICION_X / POSICION_Y (mm)	(165.5, 0), (150, 70), (106.4, 126.8)	Centro de cada cubo en la mesa
PILA_X / PILA_Y (mm)	(28.7, −163.0)	Ubicación de la pila
DIMENSION_CUBO (mm)	15	Altura de cada nivel de la pila

CUBO_Z_CENTRO (mm)	7.5	Altura del centro de un cubo apoyado en la mesa
ALTURA_APROXIMACION_MM (mm)	20	Altura sobre el punto de agarre para acercarse y subir
ALTURA_APROXIMACION_PILA_MM (mm)	6	Altura sobre el depósito de cada nivel
MARGEN_DEPOSITO_MM (mm)	2	Holgura al soltar el cubo
UMBRAL_CM (cm)	10	Distancia máxima para dar por detectado un cubo
PASO_TRAYECTORIA_MM (mm)	2	Distancia entre puntos intermedios de la trayectoria
MAX_SALTO_ARTICULAR_GRADOS (°)	10	Cambio máximo de un servo entre dos pasos
PINZA_ABIERTA / PINZA_CERRADA (°)	50 / 70	Ángulos de la pinza

1.3 Código

Extracto de Brazo_Apilado_Cinematica_ESP32_v4_wifi.ino.

Puntos de la rutina

```
Punto puntoRecogida(int n) {
        Punto p = {POSICION_X[n], POSICION_Y[n], CUBO_Z_CENTRO};
        return p;
 }

 Punto puntoAproximacion(int n) {
        Punto p = {POSICION_X[n], POSICION_Y[n], CUBO_Z_CENTRO +
ALTURA_APROXIMACION_MM};
        return p;
 }

  // Pose de escaneo: sobre el cubo, retrocediendo DESFASE_SENSOR_MM para
que el sensor quede encima
 Punto puntoEscaneo(int n) {
        Punto p = puntoAproximacion(n);
        float l = sqrtf(p.x * p.x + p.y * p.y);
        if (l > 1e-3f) {
        p.x -= DESFASE_SENSOR_MM * p.x / l;
        p.y -= DESFASE_SENSOR_MM * p.y / l;
        }
```

```
        return p;
}

// Sobre la pila: nunca más bajo que la altura de aproximación de recogida
Punto puntoSobrePila(int nivel) {
        float zNivel = CUBO_Z_CENTRO + nivel * DIMENSION_CUBO +
ALTURA_APROXIMACION_PILA_MM;
        float zTransito = CUBO_Z_CENTRO + ALTURA_APROXIMACION_MM;
        Punto p = {PILA_X, PILA_Y, fmaxf(zNivel, zTransito)};
        return p;
}

Punto puntoDeposito(int nivel) {
        Punto p = {PILA_X, PILA_Y, CUBO_Z_CENTRO + nivel *
DIMENSION_CUBO + MARGEN_DEPOSITO_MM};
        return p;
}

bool hayObjeto(float distanciaCm) {
        return distanciaCm > 0.0f && UMBRAL_CM >= distanciaCm;
}
```

Trayectoria

Se interpolan por separado el ángulo de la base, el radio horizontal y la altura. Entre dos puntos del mismo radio la garra recorre un arco, y en subidas y bajadas una recta vertical.

```
float radioDe(const Punto& p) {
        return sqrtf(p.x * p.x + p.y * p.y);
}

// Giro de la base (radianes, por el camino más corto) para ir de a hasta b
float giroBase(const Punto& a, const Punto& b) {
        if (radioDe(a) < 1e-3f || radioDe(b) < 1e-3f) return 0.0f;
        float dt = atan2f(b.y, b.x) - atan2f(a.y, a.x);
        while (dt > PI) dt -= 2.0f * PI;
        while (dt < -PI) dt += 2.0f * PI;
        return dt;
}

// Punto de la trayectoria de a hasta b para la fracción f (0 a 1)
Punto puntoEnTrayectoria(const Punto& a, const Punto& b, float f) {
        const float la = radioDe(a);
        const float lb = radioDe(b);
        float ta = atan2f(a.y, a.x);
```

```
        if (la < 1e-3f) ta = atan2f(b.y, b.x);

        const float l = la + f * (lb - la);
        const float t = ta + f * giroBase(a, b);
        Punto p = {l * cosf(t), l * sinf(t), a.z + f * (b.z - a.z)};
        return p;
}

// Longitud aproximada de la trayectoria (mm)
float longitudTrayectoria(const Punto& a, const Punto& b) {
        const float dl = radioDe(b) - radioDe(a);
        const float arco = 0.5f * (radioDe(a) + radioDe(b)) * fabsf(giroBase(a, b));
        const float dz = b.z - a.z;
        return sqrtf(dl * dl + arco * arco + dz * dz);
}

bool moverEnTrayectoria(const Punto& destino, int pausaMs) {
        if (detenerRutina) return false;

        Punto inicio;
        if (!obtenerPuntoActual(inicio)) {
        Enlace.println("[ERROR] Sin calibracion: no se puede calcular la posicion
actual.");
        return false;
        }

        int pasos = (int)ceilf(longitudTrayectoria(inicio, destino) /
PASO_TRAYECTORIA_MM);
        if (pasos < 1) pasos = 1;

        for (int i = 1; i <= pasos; i++) {
        if (paradaSolicitada()) return false;

        float t = (1.0f - cosf(PI * i / pasos)) / 2.0f;   // arranque y frenado suaves
        Punto p = puntoEnTrayectoria(inicio, destino, t);

        float servo[CANTIDAD_ARTICULACIONES];
        EstadoIK estado = calcularAngulosServo(p, servo);
        if (estado != IK_OK) {
        Enlace.print("[ERROR] Movimiento detenido: ");
          Enlace.println(describirEstadoIK(estado));
        mostrarPunto("        punto", p);
        return false;
        }
```

```
        for (int j = 0; j < CANTIDAD_ARTICULACIONES; j++) {
        if (fabsf(servo[j] - poseActual.angulos[j]) > MAX_SALTO_ARTICULAR_GRADOS) {
                Enlace.println("[ERROR] Movimiento detenido: salto articular excesivo (cambio de configuracion).");
                mostrarPunto("  punto", p);
                return false;
        }
        }

        for (int j = 0; j < CANTIDAD_ARTICULACIONES; j++) {
        poseActual.angulos[j] = servo[j];
        posicionarServo(j, servo[j]);
        }
        delay(pausaMs);
        yield();
        }
        return true;
 }
```

Transporte de un cubo y rutina principal

```
bool transportarObjeto(int posicion, int nivel) {
        nivel = constrain(nivel, 0, CANTIDAD_POSICIONES - 1);

        // 1. Alinear la garra sobre el cubo y bajar
        if (!moverEnTrayectoria(puntoAproximacion(posicion), PAUSA_PASO_LIBRE_MS)) return false;
        if (!moverEnTrayectoria(puntoRecogida(posicion), PAUSA_PASO_LIBRE_MS)) return false;

        // 2. Tomar (cerrar pinza)
        moverServoSuavemente(PINZA, PINZA_CERRADA, PAUSA_PASO_ARTICULAR_MS);
        esperarMs(PAUSA_AGARRE_MS);

        // 3. Subir con el cubo
        if (!moverEnTrayectoria(puntoAproximacion(posicion), PAUSA_PASO_TRANSPORTE_MS)) return false;

        // 4. Mover a la posición final (sobre la pila)
        if (!moverEnTrayectoria(puntoSobrePila(nivel), PAUSA_PASO_TRANSPORTE_MS)) return false;
```

```
        // 5. Bajar sobre la pila (la altura depende del nivel)
        if (!moverEnTrayectoria(puntoDeposito(nivel), PAUSA_PASO_TRANSPORTE_MS)) return false;

        // 6. Soltar el cubo
        moverServoSuavemente(PINZA, PINZA_ABIERTA, PAUSA_PASO_ARTICULAR_MS);
        esperarMs(PAUSA_AGARRE_MS);

        // 7. Subir libre de la pila
        if (!moverEnTrayectoria(puntoSobrePila(nivel), PAUSA_PASO_LIBRE_MS)) return false;

        return !detenerRutina;
 }

 ResultadoRutina ejecutarRutinaApilado() {
        if (!configuracionLista()) return RUTINA_CONFIG;
        if (!validarRutina()) return RUTINA_ALCANCE;

        estadoActual = "RUTINA";
        cubosApilados = 0;
        posicionActual = 0;
        bool error = false;

        // Pinza abierta y pose de escaneo de la posición 1 (movimiento articular desde el inicio)
        moverServoSuavemente(PINZA, PINZA_ABIERTA, PAUSA_PASO_ARTICULAR_MS);
        if (!moverAPuntoArticular(puntoEscaneo(0))) error = true;

        for (int n = 0; n < CANTIDAD_POSICIONES && !error && !detenerRutina; n++) {
        // Medir con el sensor sobre la posición actual
        posicionActual = n + 1;
        esperarMs(PAUSA_ESCANEO_MS);
        float distancia = medirDistanciaCm();
        ultimaDistanciaCm = distancia;

        if (hayObjeto(distancia)) {
        if (!transportarObjeto(n, cubosApilados)) {
                error = true;
                break;
        }
```

```
        cubosApilados++;
        }

        // Pasar a la siguiente posición (tanto si se transportó un objeto como si no)
        if (n < CANTIDAD_POSICIONES - 1) {
        if  (!moverEnTrayectoria(puntoEscaneo(n  +  1),  PAUSA_PASO_LIBRE_MS))
error = true;
        }
        }

        if (detenerRutina) return RUTINA_DETENIDA;
        if (error) return RUTINA_ERROR;

        volverAInicio();
        if (detenerRutina) return RUTINA_DETENIDA;
        return RUTINA_OK;
}
```

8. Anexo II: Desarrolla del algoritmo D-H

2.1 Parámetros y convención

El brazo tiene tres articulaciones rotacionales (base q1, hombro q2, codo q3) más un tramo fijo hasta el punto de agarre de la garra. La pinza se mueve aparte y no entra en la cadena. Se usa la convención D-H estándar: cada eslabón se describe con cuatro parámetros (θ, d, a, α) y su transformación es una rotación en z, un desplazamiento en z, un desplazamiento en x y una rotación en x.

$$A_i = R_z(\theta_i)\,T_z(d_i)\,T_x(a_i)\,R_x(\alpha_i) =$$
$$\begin{bmatrix}$$
$$\cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\$$
$$\sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\$$
$$0 & \sin\alpha_i & \cos\alpha_i & d_i \\$$
$$0 & 0 & 0 & 1$$
$$\end{bmatrix}$$

Medidas del robot (las mismas del código, en config):

Símbolo	Valor	Significado
d1	129.5 mm	Altura del eje del hombro sobre la mesa
a2	70.53 mm	Eje del hombro al eje del codo

a3	50.035 mm	Eje del codo al inicio del tramo de la garra
dx	100.3 mm	Tramo de la garra, a lo largo del antebrazo
dh	15 mm	El punto de agarre queda esto por debajo de la línea del antebrazo
a4	101.415 mm	Tramo fijo de la garra: $\sqrt{(dx^2 + dh^2)}$
φ	8.506°	Inclinación del tramo de la garra: atan2(dh, dx)

La garra tiene un desfase en distancia (dx) y en altura (dh) respecto del antebrazo, por eso el último eslabón se trata como un tramo fijo de longitud a4 inclinado −φ.

Los ángulos de articulación q se convierten a ángulo de servo con servo = offset + signo · q. Offsets {90, 70, 77}° y signos {+1, −1, +1} para base, hombro y codo.

2.2 Tabla D-H y matriz de transformación

La tabla tiene cuatro filas: las tres articulaciones y el tramo fijo de la garra.

i	θi	di (mm)	ai (mm)	αi
1 (base)	q1	129.5	0	90°
2 (hombro)	q2	0	70.53	0°
3 (codo)	q3	0	50.035	0°
4 (garra, fija)	−φ = −8.506°	0	101.415	0°

La posición del punto de agarre se obtiene multiplicando las cuatro matrices:

$$T_{04} = A_1\,A_2\,A_3\,A_4$$

Con c1 = cos q1, s1 = sin q1, c23 = cos(q2 + q3) y s23 = sin(q2 + q3), la posición (x, y, z) de T04 se reduce a:

$$\begin{aligned}
R &= a_2 \cos q_2 + (a_3 + d_x)\cos(q_2+q_3) + d_h \sin(q_2+q_3) \\
Z &= a_2 \sin q_2 + (a_3 + d_x)\sin(q_2+q_3) - d_h \cos(q_2+q_3) \\
x &= c_1 R, \qquad y = s_1 R, \qquad z = d_1 + Z
\end{aligned}$$

R es el radio horizontal medido desde el eje de la base y Z la altura sobre el hombro.

2.3 Algoritmo

Cinemática directa. Dados q1, q2, q3 (en grados):

1. Calcular A1, A2, A3 y A4 con la matriz D-H estándar y los parámetros de la tabla.
2. Acumular T1 = A1, T2 = T1·A2, T3 = T2·A3 y T4 = T3·A4.
3. La posición de cada articulación es la cuarta columna de cada T: T1 es el hombro, T2 el codo, T3 la muñeca y T4 el punto de agarre. Estos cuatro puntos son los que el robot envía como mensaje DH.

En el código es cadenaDH() y cinematicaDirecta().

Cinemática inversa (codo arriba). Dado el punto de agarre (x, y, z):

1. Base: q1 = atan2(y, x). Radio l = √(x² + y²) y altura relativa al hombro zr = z − d1.
2. Distancia del hombro al punto: h = √(l² + zr²). Angúlo de elevación: α = atan2(zr, l).
3. Del codo al punto de agarre hay una distancia fija L3 = √((a3 + dx)² + dh²) = 151.08 mm, con una inclinación δ = atan2(−dh, a3 + dx) = −5.70°. Con L2 = a2, por el teorema del coseno:

\cos\beta = \frac{L_2^2 + h^2 - L_3^2}{2\,L_2\,h}

4. Si |cos β| > 1 el punto está fuera de alcance. Si no: q2 = α + β (codo arriba).
5. Dirección del codo al punto: ψ = atan2(zr − L2 sin q2, l − L2 cos q2). Codo: q3 = ψ − q2 − δ.
6. Convertir cada q a ángulo de servo (servo = offset + signo · q) y comprobar que está dentro de los límites del servo. Si no, el punto se rechaza.

En el código es cinematicaInversa() y calcularAngulosServo().

Con tres articulaciones la inclinación de la garra queda determinada por la posición: no se puede elegir una garra vertical sin una cuarta articulación de muñeca. La rutina valida antes de moverse que todos los puntos y los tramos entre ellos cumplan estas condiciones (validarRutina()).

2.4 Resultados

Los resultados se calcularon con las mismas fórmulas del firmware (doble precisión) y los parámetros de 2.1. El punto de agarre de la pose inicial coincide con lo que mide el propio robot en su mensaje DH.

Cinemática directa: pose inicial

Servos en (80°, 80°, 80°), es decir q = (−10°, −10°, 3°):

Punto	x (mm)	y (mm)	z (mm)
Hombro (T1)	0.00	0.00	129.50
Codo (T2)	68.40	−12.06	117.25
Muñeca (T3)	117.31	−20.69	111.15
Garra (T4)	213.55	−37.65	84.04

T_{04} =
\begin{bmatrix}
0.9490 & 0.2633 & -0.1736 & 213.55 \\
-0.1673 & -0.0464 & -0.9848 & -37.65 \\
-0.2673 & 0.9636 & 0 & 84.04 \\
0 & 0 & 0 & 1
\end{bmatrix}

La fórmula cerrada de 2.2 da el mismo resultado que el producto de matrices. Por ejemplo, con q = (30°, 40°, −25°) ambos métodos entregan (175.91, 101.56, 199.26) mm.

Cinemática inversa: puntos de la rutina

Los puntos se resolvieron con la configuración del archivo v4 (cubo de 15 mm). Cada solución se verificó aplicando la cinemática directa: el error de posición es menor que 10^{-12} mm en todos los casos.

Punto	(x, y, z) mm	q1, q2, q3 (°)	Servos base, hombro, codo (°)	Límites
Recogida 1	(165.5, 0, 7.5)	0.00, −3.76, −41.52	90.0, 73.8, 35.5	cumple
Recogida 2	(150, 70, 7.5)	25.02, −3.78, −41.49	115.0, 73.8, 35.5	cumple
Recogida 3	(106.4, 126.8, 7.5)	50.00, −3.78, −41.49	140.0, 73.8, 35.5	cumple
Depósito nivel 1	(28.7, −163, 9.5)	−80.01, −2.07, −43.26	10.0, 72.1, 33.7	cumple
Depósito nivel 2	(28.7, −163, 24.5)	−80.01, 9.55, −54.42	10.0, 60.5, 22.6	cumple
Depósito nivel 3	(28.7, −163, 39.5)	−80.01, 19.87, −63.14	10.0, 50.1, 13.9	cumple
Sobre la pila, nivel 3	(28.7, −163, 43.5)	−80.01, 22.47, −65.16	10.0, 47.5, 11.8	cumple

Límites de servo usados: base 0–180°, hombro 10–140°, codo 10–170°. La restricción que más se acerca es el codo en el tercer nivel de la pila (11.8° frente al mínimo de 10°), y es la que limita qué tan cerca de la base se puede poner la pila.

En la pila la altura z sube 15 mm por nivel (el tamaño del cubo), q1 se mantiene en −80.01° y solo cambian el hombro y el codo.